

Deep Learning +

Graph +

Link Prediction +

Autoencoder +

Data

Link prediction methods overview



Jakub Skalski

Follow

6 min read · Nov 8, 2024



10



AdBlock



Your connection isn't as private as you think

AdBlock VPN encrypts your traffic and hides your IP — on any network, any device.

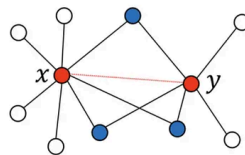
Try it free for 7 days.

Protect my connection

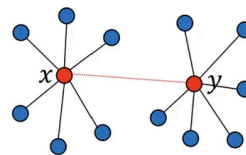
The **link prediction problem** is a fundamental task in network analysis and machine learning, aiming to predict the likelihood of a connection (or “link”) between two nodes in a network. Link prediction has applications in various domains, such as social network analysis, recommender systems, bioinformatics, and knowledge graphs.

Local heuristics

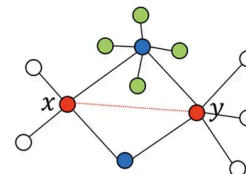
This class of heuristics considers only the local subgraphs of the nodes in question. Some of the more popular ones are **common neighbors**, **preferential attachment** and **Adamic-Adar**. Here $\Gamma(v)$ denotes the neighbors of v .



common neighbors (CN):
 $|\Gamma(x) \cap \Gamma(y)|$



preferential attachment (PA):
 $|\Gamma(x)| \cdot |\Gamma(y)|$



Adamic-Adar (AA):
 $\frac{1}{\sum_{z \in \Gamma(x) \cap \Gamma(y)} \frac{1}{\log |\Gamma(z)|}}$

Global Heuristics

These heuristics, also known as high order heuristics, make their decision based on the entire network.

Katz index

This index calculates a score based on all possible paths between two nodes, where longer paths contribute less to the score than shorter paths. This captures the idea that while two nodes might not be directly connected, they could still be “close” in terms of the network structure.

$$f_{\text{Katz}}(x, y) = \sum_{l=1}^{\infty} \beta^l |\text{walks}|$$

Where the number of l -walks can be computed as an adjacency matrix raised to the l -th power.

Rooted PageRank

Let π_x be a stationary distribution representing the long-term behavior of a random walker on a graph that follows a set of rules:

- move to one of its neighboring nodes with probability α
- return to x with probability $1-\alpha$

Therefore, $[\pi_x]_y$ essentially estimates how likely it is to reach node y from node x , which can be interpreted as the strength or likelihood of a connection (link) from x to y .

$$f_{\text{RPR}}(x, y) = [\pi_x]_y + [\pi_y]_x$$

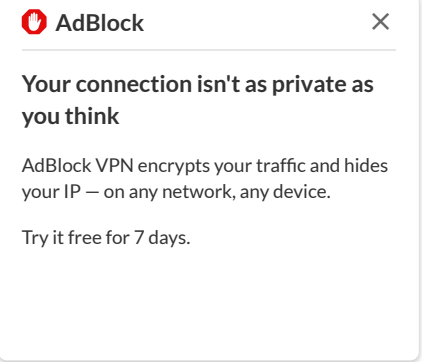
Embedding methods

The second class of traditional link prediction methods, known as embedding methods (or latent-feature methods), compute hidden node representations. These features, derived from network structures like adjacency or Laplacian matrices, are not directly interpretable.

Matrix Factorization

Matrix factorization decomposes the observed adjacency matrix A into the product of a low-rank latent-embedding matrix Z and its transpose, allowing for the edges reconstruction.

$$\hat{A}_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j$$



The latent embeddings are learned by minimizing the mean-squared error between the reconstructed and true adjacency matrices.

$$\mathcal{L} = \frac{1}{|\mathcal{E}|} \sum_{(i,j) \in \mathcal{E}} (A_{i,j} - \hat{A}_{i,j})^2$$

AdBlock

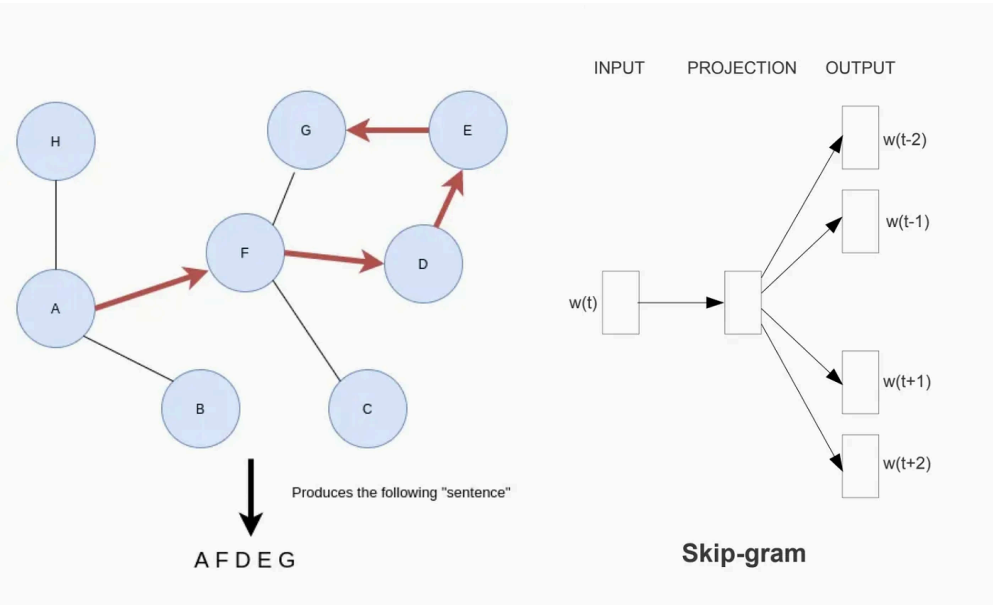
Your connection isn't as private as you think

AdBlock VPN encrypts your traffic and hides your IP — on any network, any device.

Try it free for 7 days.

Node2Vec

The Node2Vec algorithm is a popular method for generating node embeddings in a graph. It combines techniques from **random walks** and **word2vec** to learn representations of nodes in a way that preserves the graph's structure and properties. The general idea of the algorithm is to first gather likely sequences of vertices by performing random walks and then feed them to a skip-gram model to learn embeddings.



Get Jakub Skalski's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

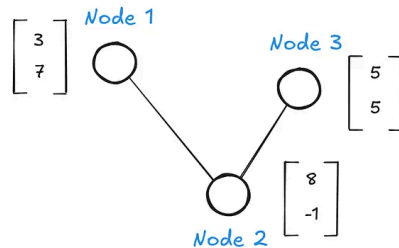
☐ Remember me for faster sign in

Graph neural networks

The main advantage of graph neural networks is their ability to train on both the graph structure (adjacencies) and its contents (node features).

Graph Convolutional Networks (GCN)

Graph Convolutional Networks consume an adjacency-like matrix of node features X to produce embeddings:



Example of a GCN from literature

$$\text{GCN}(X, A) = \tilde{A} \text{ReLU}(\tilde{A} X W_0) W_1$$

$$\tilde{A} = D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$$

aggregation + combination

$$\begin{matrix} \text{Node 1} \\ \text{Node 2} \\ \text{Node 3} \end{matrix} \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 3 & 7 \\ 8 & -1 \\ 5 & 5 \end{bmatrix} \begin{bmatrix} w_1 \\ \dots \\ w_n \end{bmatrix}^T = \begin{bmatrix} \text{Node1} + \text{Node2} \\ \text{Node1} + \text{Node2} + \text{Node3} \\ \text{Node2} + \text{Node3} \end{bmatrix} \begin{bmatrix} w_1 \\ \dots \\ w_n \end{bmatrix}^T$$

$A \quad X \quad W \quad H \quad W$

Illustration of a single layer GCN computations

The methods discussed here all rely on the GCN architecture. We differentiate two types of neural network paradigms for link prediction:

- **node-based methods** aggregate the pairwise node representations learned by a GNN as the link representation.
- **sub-graph methods** extract a local subgraph around each link and use the subgraph representation learned by a GNN as the link representation.

Variational Graph AutoEncoder (VGAE)

Variational Graph AutoEncoders ([Kipf and Welling, 2016](#)) are node-based methods. They combine the main ideas of regular variational autoencoders and GCNs. The use of GCNs allows the network to efficiently process the input graph while the VAEs combine the robustness to the sparsity of the data (the adjacency matrices of most of the naturally formed graphs tend to be sparse — most of the edges are negative) and a nonlinear dimensionality reduction capability.

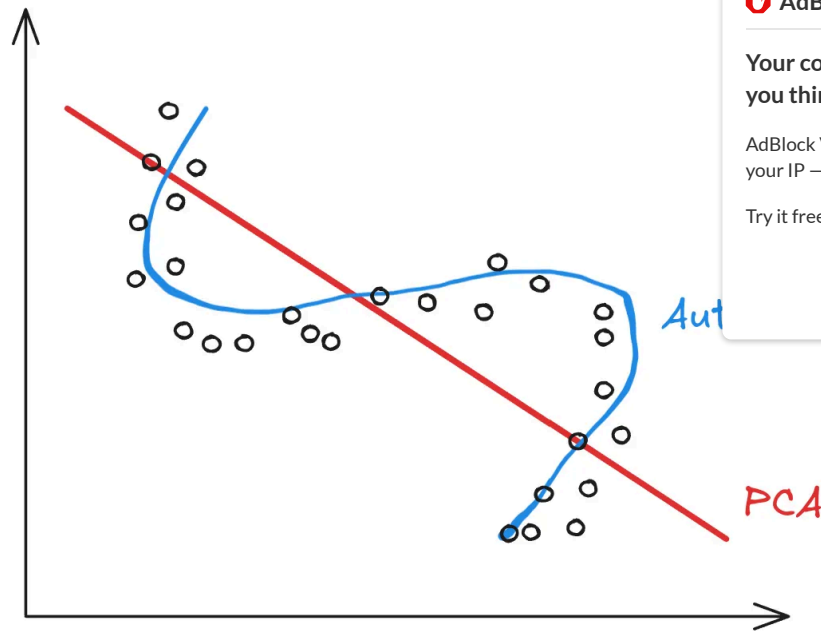
AdBlock
×

Your connection isn't as private as you think

AdBlock VPN encrypts your traffic and hides your IP — on any network, any device.

Try it free for 7 days.

Linear vs nonlinear dimensionality reduction



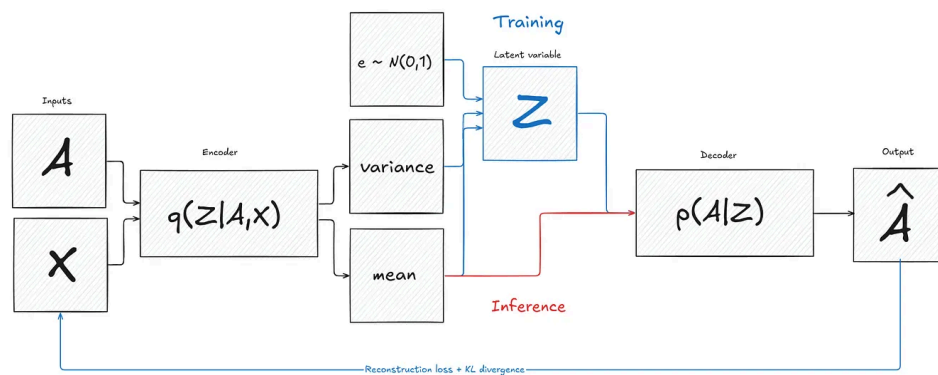
AdBlock

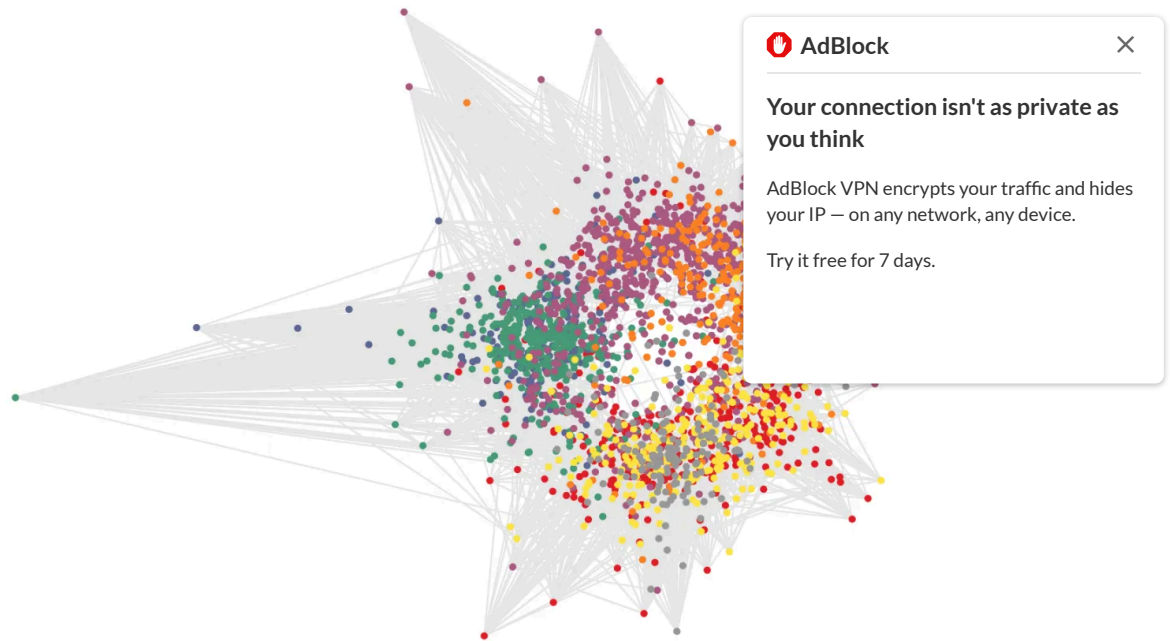
Your connection isn't as private as you think

AdBlock VPN encrypts your traffic and hides your IP — on any network, any device.

Try it free for 7 days.

The following VGAE architecture in uses simple dot product as its decoder since the similarity between the embeddings is easily expressed as the probability of a link between them.

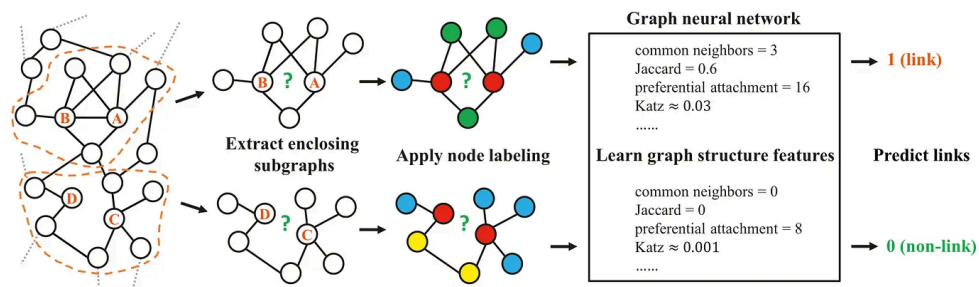




Latent space of unsupervised VGAE model trained on Cora citation network dataset ([Kipf and Welling, 2016](#)).

SEAL

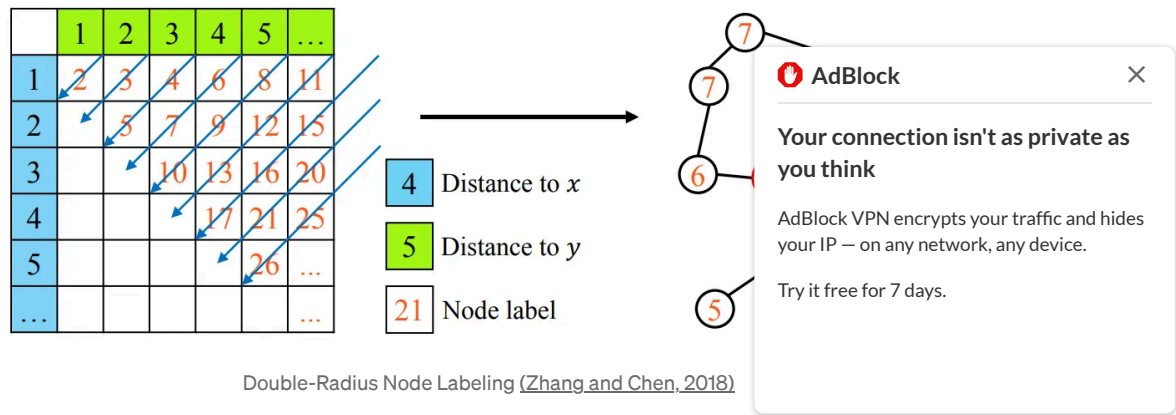
SEAL ([Zhang and Chen, 2018](#)) framework (learning from Subgraphs, Embeddings, and Attributes for Link prediction) belongs to the **sub-graph methods** paradigm.



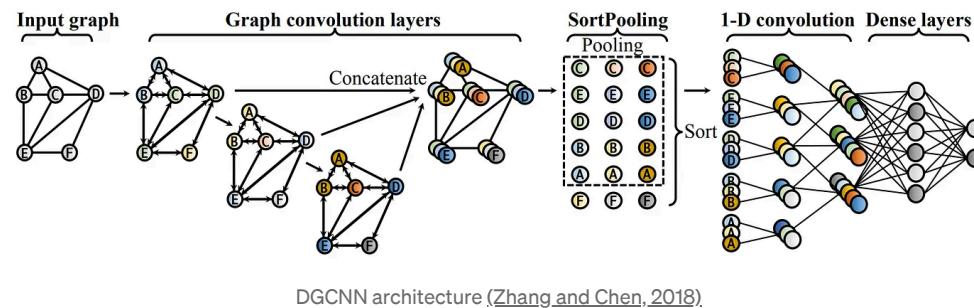
SEAL framework (Graph Neural Networks: Foundations, Frontiers, and Applications, 2022, Lingfei Wu et al).

The SEAL framework can be divided into 3 consecutive steps:

1. For each pair of nodes u and v use k-hop neighborhood algorithm (nodes within k steps from u and v) to extract the subgraph.
2. Give an integer label to each node in the subgraph as its additional feature to differentiate nodes of different roles within a subgraph using Double Radius Node Labeling algorithm (iteratively assign larger labels to nodes with a larger radius w.r.t. both center nodes).



3. Feeds the enclosing subgraphs as well as their new node feature vectors into a graph-level GNN, DGCNN (Zhang et al, 2018g), to learn a graph classification function. The groundtruth of each subgraph is whether the two center nodes really have a link.



To train this GNN, SEAL randomly samples N existing links from the network as positive training links, and samples an equal number of unobserved links (random node pairs) as negative training links.

Comparison and conclusions

The following is the comparison of the SEAL framework with graph structure link prediction methods, including eight common heuristics (e.g., common neighbors, Jaccard, and Adamic-Adar), and two heuristic learning methods: the Weisfeiler-Lehman kernel and WLNLM, both of which use enclosing subgraphs. Methods focused on knowledge graphs or recommendation systems are omitted. To ensure comparability, SEAL is restricted to graph structure only (uses no none of the original node features, i.e X matrix is an identity), with the hop number h chosen from $\{1, 2\}$, as larger values don't improve performance and may lead to excessively large subgraphs.

Data	CN	Jaccard	PA	AA	RA	Katz	PR	SR	ENS	WLK	WLNLM	SEAL
USAir	93.80±1.22	89.79±1.61	88.84±1.45	95.06±1.03	95.77±0.92	92.88±1.42	94.67±1.08	78.89±2.31	88.96±1.44	96.63±0.73	95.95±1.10	96.62±0.72
NS	94.42±0.95	94.43±0.93	68.65±2.03	94.45±0.93	94.45±0.93	94.85±1.10	94.89±1.08	94.79±1.08	97.64±0.25	98.57±0.51	98.61±0.40	98.85±0.47
PB	92.04±0.35	87.41±0.39	90.14±0.45	92.36±0.34	92.46±0.37	92.92±0.35	93.54±0.41	77.08±0.80	90.15±0.45	97.07±0.57	97.07±0.57	97.07±0.57
Yeast	89.37±0.61	89.32±0.60	82.20±1.02	89.43±0.62	89.45±0.62	92.24±0.61	92.76±0.55	91.49±0.57	82.36±1.02	97.07±0.57	97.07±0.57	97.07±0.57
C.ele	85.13±1.61	80.19±1.64	74.79±2.04	86.95±1.40	87.49±1.41	86.34±1.89	90.32±1.49	77.07±2.00	74.94±2.04	97.07±0.57	97.07±0.57	97.07±0.57
Power	58.80±0.88	58.79±0.88	44.33±1.02	58.79±0.88	58.79±0.88	65.39±1.59	66.00±1.59	76.15±1.06	79.52±1.78	97.07±0.57	97.07±0.57	97.07±0.57
Router	56.43±0.52	56.40±0.52	47.58±1.47	56.43±0.51	56.43±0.51	38.62±1.35	38.76±1.39	37.40±1.27	47.58±1.48	97.07±0.57	97.07±0.57	97.07±0.57
E.coli	93.71±0.39	81.31±0.61	91.82±0.58	95.36±0.34	95.95±0.35	93.50±0.44	95.57±0.44	62.49±1.43	91.89±0.58	97.07±0.57	97.07±0.57	97.07±0.57

Comparison with heuristic methods (Zhang and Chen, 2018)

Methods that learn from enclosing subgraphs (like WLK, WLNLM) outperform predefined heuristics, showing that learned heuristics use network properties more effectively. SEAL leads among learned methods, highlighting the advantage of GNNs for feature learning. Even in cases where heuristics fail, such as with Power and Router networks, learning-based methods perform well, suggesting they can reveal new, effective “heuristics” for complex networks.

Data	MF	SBM	N2V	LINE	SPC	VGAE	SEAL
USAir	94.08±0.80	94.85±1.14	91.44±1.78	81.47±10.71	74.22±3.11	89.28±1.99	97.09±0.70
NS	74.55±4.34	92.30±2.26	91.52±1.28	80.63±1.90	89.94±2.39	94.04±1.64	97.71±0.93
PB	94.30±0.53	93.90±0.42	85.79±0.78	76.95±2.76	83.96±0.86	90.70±0.53	95.01±0.34
Yeast	90.28±0.69	91.41±0.60	93.67±0.46	87.45±3.33	93.25±0.40	93.88±0.21	97.20±0.64
C.ele	85.90±1.74	86.48±2.60	84.11±1.27	69.21±3.14	51.90±2.57	81.80±2.18	89.54±2.04
Power	50.63±1.10	66.57±2.05	76.22±0.92	55.63±1.47	91.78±0.61	71.20±1.65	84.18±1.82
Router	78.03±1.63	85.65±1.93	65.46±0.86	67.15±2.10	68.79±2.42	61.51±1.22	95.68±1.22
E.coli	93.76±0.56	93.82±0.41	90.82±1.49	82.38±2.19	94.92±0.32	90.81±0.63	97.22±0.28

Comparison with embedding and other GNN methods (Zhang and Chen, 2018)

SEAL outperforms six embedding (latent feature) methods, including matrix factorization, node2vec, and VGAE. While VGAE uses a node-level GNN to reconstruct the network, SEAL employs a graph-level GNN to classify subgraphs, making it more effective for link prediction. By integrating node2vec embeddings, SEAL enhances its performance over methods that rely solely on latent features, as it captures additional information from local graph structures. However, joint learning with node2vec does not always boost SEAL’s performance, suggesting that network embeddings alone may miss critical local structural details.

Deep Learning +

Graph +

Link Prediction +

Autoencoder +

Data Science +



Written by Jakub Skalski

2 followers · 1 following

Follow

 AdBlock ×

Your connection isn't as private as you think

AdBlock VPN encrypts your traffic and hides your IP — on any network, any device.

Try it free for 7 days.

 AdBlock ×

Your connection isn't as private as you think

AdBlock VPN encrypts your traffic and hides your IP — on any network, any device.

Try it free for 7 days.

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Store](#) [Privacy](#) [Rules](#) [Terms](#) [Text](#)